

基于循环神经网络（RNN）的字符级语言模型实现

1 实验目的

- 深入理解循环神经网络（RNN）及其变体 **LSTM** 和 **GRU** 的数学原理。
- 掌握使用 PyTorch 从底层（Cell 级别）构建深度学习模型的方法。
- 实现一个字符级预测模型，能够根据给定文本序列预测下一个字符。

2 背景知识与数学原理

2.1 传统 RNN：序列建模的基石

传统的神经网络（如 MLP）假设输入之间是相互独立的，但在处理序列数据（如文本）时，前后的上下文具有强烈的相关性。RNN 通过在隐藏层引入反馈环路来实现“记忆”。

底层原理

在每一个时间步 t ，RNN 单元执行以下操作：

1. 特征融合：同时接收当前的输入 x_t 和上一时刻的隐状态 h_{t-1} 。
2. 线性映射：通过权重矩阵 W_{ih} 处理输入，通过 W_{hh} 处理旧记忆。
3. 非线性激活：使用 $\tanh$ 函数将结果压缩到 $[-1, 1]$ 之间，生成新的记忆 h_t 。

数学表达

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh}) \quad (1)$$

致命缺陷：梯度消失（**Vanishing Gradient**）

在反向传播（BPTT）时，梯度需要跨越多个时间步。由于 $\tanh$ 导数小于 1 且权重矩阵的累积乘法，梯度会随序列长度增加而呈指数级衰减。这导致标准 RNN 难以捕捉长距离的依赖关系。

2.2 LSTM：长短期记忆的“闸门”控制

为了解决梯度消失问题，LSTM 引入了细胞状态（Cell State, c_t ）。它像一条贯穿所有时间步的“传送带”，信息可以在上面流传而不被轻易修改。

核心机制：门控 (Gates)

LSTM 通过三个“门”来控制信息流，这些门使用 Sigmoid 激活函数将数值压缩在 0 到 1 之间，表示允许通过的信息比例。

1. 遗忘门 (Forget Gate, f_t) 公式:

$$f_t = \sigma(W_{if}x_t + W_{hf}h_{t-1} + b_f) \quad (2)$$

2. 输入门 (Input Gate, i_t) 与细胞门 (g_t)

$$i_t = \sigma(W_{ii}x_t + W_{hi}h_{t-1} + b_i) \quad (3)$$

$$g_t = \tanh(W_{ig}x_t + W_{hg}h_{t-1} + b_g) \quad (4)$$

3. 细胞状态更新

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (5)$$

4. 输出门 (Output Gate, o_t)

$$o_t = \sigma(W_{io}x_t + W_{ho}h_{t-1} + b_o) \quad (6)$$

$$h_t = o_t \odot \tanh(c_t) \quad (7)$$

2.3 GRU: 更简洁、更高效的演进

GRU 是 LSTM 的优化版本，它将隐藏状态和细胞状态合并，并将门控数量减为两个，从而提高了计算效率。

核心逻辑

1. 重置门 (Reset Gate, r_t)

$$r_t = \sigma(W_{ir}x_t + W_{hr}h_{t-1} + b_r) \quad (8)$$

2. 更新门 (Update Gate, z_t)

$$z_t = \sigma(W_{iz}x_t + W_{hz}h_{t-1} + b_z) \quad (9)$$

3. 最终状态更新

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1} \quad (10)$$

2.4 总结与对比

3 数据处理模块 (data.py)

在字符级语言模型中，模型无法直接处理原始文本字符串。data.py 的核心任务是将非结构化的文本数据转换为模型可计算的数值张量 (Tensors)，并实现高效的数据加载机制。

特性	RNN	LSTM	GRU
内部状态	仅 h_t	h_t 与 c_t	仅 h_t
门控数量	无	3	2
梯度传导	容易消失	优秀	优秀
计算复杂度	最低	最高	中等
参数量	最小	最大	较小

3.1 字符级编码 (Character-level Encoding)

由于我们进行的是字符级建模，词表 (Vocabulary) 由文本中出现的所有唯一字符组成。

- 构建词表：遍历 `train.txt` 中的所有字符，通过集合 (Set) 去重，确定词表大小 `input_size` (即 `n_characters`)。
- 字符映射字典：
 - `char_to_index`: 将每个字符映射为一个唯一的整数 (0 到 $n - 1$)。例如：
`'a' → 0, 'b' → 1`。
 - `index_to_char`: 将整数索引还原为原始字符。这在模型推理阶段 (即生成文本时) 至关重要，用于将模型的概率输出转换回可读文本。
- **One-hot 编码 / Embedding**: 在 `model_new.py` 中, 输入的索引会通过 `nn.Embedding` 层转换为稠密向量，从而捕获字符间的语义关系。

3.2 数据采样策略 (Random Chunking)

为了提高训练效率并满足循环神经网络对序列输入的要求，我们采用随机采样机制：

1. 加载语料：读取整个 `train.txt` 文件并将其转换为长索引张量。
2. 随机分块 (Chunking):
 - 每次训练迭代时, 从长文本中随机选择一个起始点, 截取长度为 `chunk_len+1` 的字符序列。
 - 这种方法可以确保模型在多次训练中能见到不同的上下文组合，增强泛化能力。
3. 构造训练对 (Input & Target):
 - **Input (x)**: 序列中的前 n 个字符 (第 1 到第 n 个), 被送入 CharRNN 处理。
 - **Target (y)**: 序列中的后 n 个字符 (第 2 到第 $n+1$ 个), 作为预测目标标签。
 - 逻辑: 当模型输入第 t 个字符时, 其目标是输出第 $t+1$ 个字符的概率分布。

3.3 Batch 构建与维度变换

- 批处理 (Batching): 将多个随机 Chunk 组合在一起形成形状为 (`batch_size`, `seq_len`) 的张量。
- 维度对齐:

- 在 `CharRNN.forward` 中, 输入的 $(batch, seq_len)$ 首先通过 `Embedding` 变为 $(batch, seq_len, hidden_size)$ 。
- 为了配合自定义的 `StackedRNN` 逐时间步处理, 数据会被转置为 $(seq_len, batch, hidden_size)$ 。
- 最终, 为了计算交叉熵损失 (`CrossEntropyLoss`), 输出会展平为二维矩阵, 每一行代表一个时间步的预测结果。

4 网络结构模块介绍

在 `model_new.py` 中, 我们采用了层次化的设计架构, 将复杂的网络拆解为 **Cell** (单元)、**Stacked** (堆叠层) 和 **Interface** (整体接口) 三部分。

4.1 RNN (循环神经网络)

- 结构特点: 最基础的循环结构, 每一时刻的隐状态 h_t 由当前输入 x_t 和前一时刻隐状态 h_{t-1} 共同决定。
- 实现细节: 在 `RNNCell` 中, 通过 `nn.Linear` 实现 W_{ih} 和 W_{hh} 的线性变换, 并使用 `torch.tanh` 作为激活函数。

4.2 LSTM (长短期记忆网络)

- 结构特点: 引入细胞状态 c 和三个门控结构:
 - 遗忘门: 决定丢弃多少旧信息。
 - 输入门: 决定存入多少新信息。
 - 输出门: 决定从细胞状态中输出多少信息到 h_t 。
- 实现细节: 在 `LSTMCell` 中, 四个门 (输入、遗忘、细胞、输出) 的权重被合并在一个大线性层中以提高计算效率, 随后使用 `.chunk(4)` 进行拆分处理。

4.3 GRU (门控循环单元)

- 结构特点: LSTM 的简化变体, 将遗忘门和输入门合并为更新门 (Update Gate), 并引入重置门 (Reset Gate)。
- 实现细节:
 - 通过重置门控制前一时刻隐状态 h_{t-1} 对当前候选状态的影响。
 - 使用更新门在旧状态和新候选状态之间进行插值。
 - 参数量比 LSTM 少约 1/3, 在中小规模数据集上通常表现更优。

4.4 模块化堆叠逻辑 (Stacked Modules)

为了构建深层网络, 模型实现了 `StackedRNN/LSTM/GRU` 类:

- 纵向堆叠: 使用 `nn.ModuleList` 管理多个 `Cell`, 将第 i 层的输出作为第 $i+1$ 层的输入。

- 正则化：在除最后一层外的各层之间应用 **Dropout**，防止模型过拟合。
- 状态传递：负责在时间步之间正确传递和更新每一层的隐状态（或初始状态）。

5 实验任务

在本次实验中，你将基于 `model_new.py` 中的框架完成以下任务：

5.1 任务 A：完善单步 Cell 实现

你需要补全以下三个类中的 `forward` 方法。注意，我们不直接调用 `nn.LSTM`，而是手动实现它们的逻辑：

1. **RNNCell**：实现最基础的 `tanh` 更新逻辑。
2. **LSTMCell**：实现四门机制。提示：代码中已将四个门的权重合并在一个线性层中，你需要使用 `.chunk(4, dim=1)` 拆分它们。
3. **GRUCell**：实现重置门、更新门以及候选状态的组合。

5.2 任务 B：构建多层堆叠结构（Stacked Model）

当层数 $n_layers > 1$ 时，前一层的输出将作为后一层的输入：

- 实现 `StackedRNN` / `StackedLSTM` / `StackedGRU` 的循环逻辑。
- 在层与层之间添加 **Dropout** 提高模型的泛化能力（最后一层除外）。

5.3 任务 C：字符级模型整合（CharRNN）

完成 `CharRNN` 类的定义：

- **Embedding** 层：将字符索引转换为向量。
- **RNN** 层：调用你实现的 `Stacked` 系列类。
- **Decoder** 层：将隐状态映射回字符词表大小的维度。

5.4 任务 D：优化训练策略

在 `train.py` 中，通过引入现代训练技巧来提升 RNN 模型的稳定性和收敛速度。你需要补全以下两个关键部分：

1. 学习率调度（Learning Rate Scheduler）

- 目标：在训练后期降低学习率，使模型能够更精细地收敛到局部最优解。
- 实现逻辑：使用 `torch.optim.lr_scheduler.StepLR`。
在训练总周期的 30% 处 (`step_size`) 将学习率减半 ($\gamma = 0.5$)。

2. 梯度裁剪 (Gradient Clipping)

- 目标：解决循环神经网络 (RNN) 中常见的梯度爆炸问题，限制梯度范数。
- 实现逻辑：在 `loss.backward()` 之后、`optimizer.step()` 之前，使用 `torch.nn.utils.clip_grad_norm_` 将模型参数的梯度范数限制在 5.0 以内。

6 实验评估指标

在模型评估阶段 (`eval.py`)，我们通过以下三个核心指标来量化模型在验证集或测试集上的表现：

6.1 平均交叉熵损失 (Average Loss)

交叉熵损失用于衡量模型输出概率分布与真实标签之间的距离。

- 计算逻辑：累加每个 batch 的 `nn.CrossEntropyLoss`，最后除以总的迭代步数 `total_steps`。

$$\text{Loss}_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N \text{Loss}_i \quad (11)$$

6.2 困惑度 (Perplexity, PPL)

困惑度衡量语言模型预测样本的概率。对于语言模型而言，困惑度本质上是交叉熵的指数形式。

- 计算逻辑：对平均损失值取自然指数 `math.exp(avg_loss)`。

$$\text{PPL} = \exp(\text{Loss}_{\text{avg}}) \quad (12)$$

意义：PPL 越小，代表模型生成的序列越自然，预测越准确。

6.3 准确率 (Accuracy)

准确率反映了模型预测出的最高概率字符与实际目标字符的一致性。

- 计算逻辑：统计所有预测正确的 Token 数量 `total_correct`，除以样本总数 `total_tokens`。

$$\text{Acc} = \frac{N_{\text{correct}}}{N_{\text{total}}} \times 100\% \quad (13)$$

7 实验要点与提示

1. 数据维度控制：
 - 在 forward 过程中，注意 `x_seq` 的维度通常是 $(seq_len, batch, input_size)$ 。
 - CharRNN 接口为了兼容性，在输入后使用了 `permute(1, 0, 2)` 来匹配手写 RNN 的时间步优先顺序。
2. 隐藏状态初始化：
 - 每次处理新的 Batch 前需调用 `init_hidden`。
 - 对于 LSTM，需要同时初始化初始隐藏状态和细胞状态： h_0 和 c_0 。
3. 参数共享：
 - 在 LSTMCell 中， W_{ih} 和 W_{hh} 包含了四个门的所有参数，输出维度为： $4 \times hidden_size$ 。

8 思考题

1. 梯度问题：为什么 LSTM 的细胞状态 c_t 的更新方式（加法运算）比标准 RNN（乘法加激活）更能缓解梯度消失？
2. 结构对比：GRU 相比于 LSTM 缺少了哪个门？这对于计算效率有什么影响？
3. 维度转换：在 `CharRNN.forward` 中，为什么要将输出展平为

$$(batch_size \times seq_len, hidden_size)$$

后再送入 Decoder？